CS 378: Systems for Generative AI

Assignment 3: LLM Serving with Vidur

Name, EID: Kevin Babu (kb45994), Manas Agrawal (ma64625)

Model and hardware throughout: Llama-3-8B on one simulated A100-80GB, replaying `traces/azure_conversation.csv` (19,366 requests).

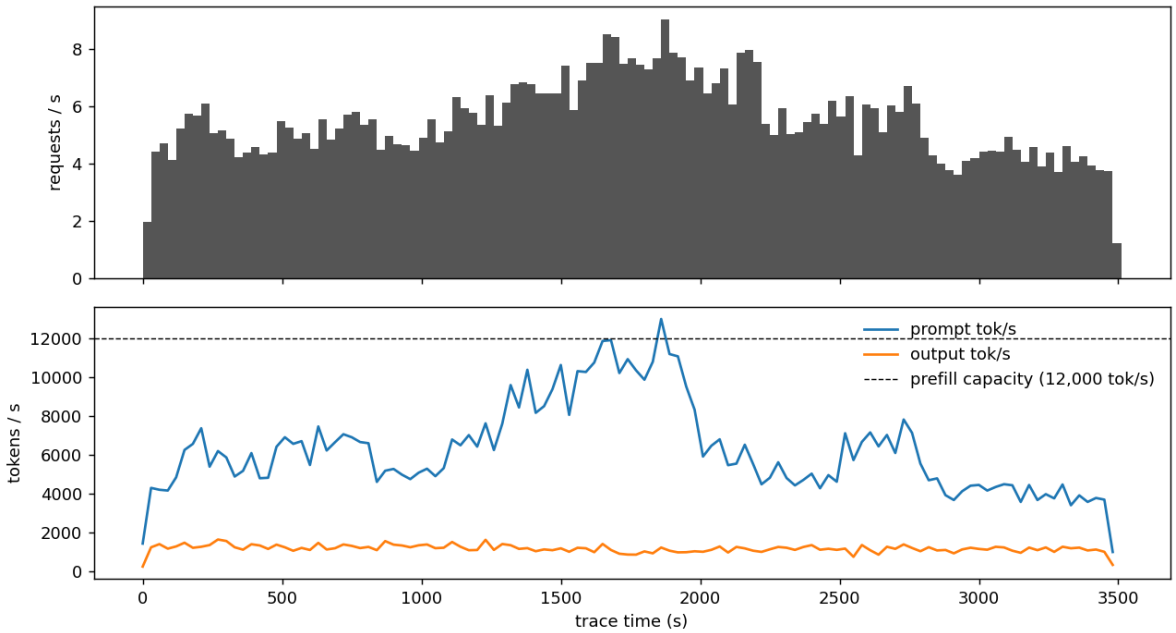
Test results

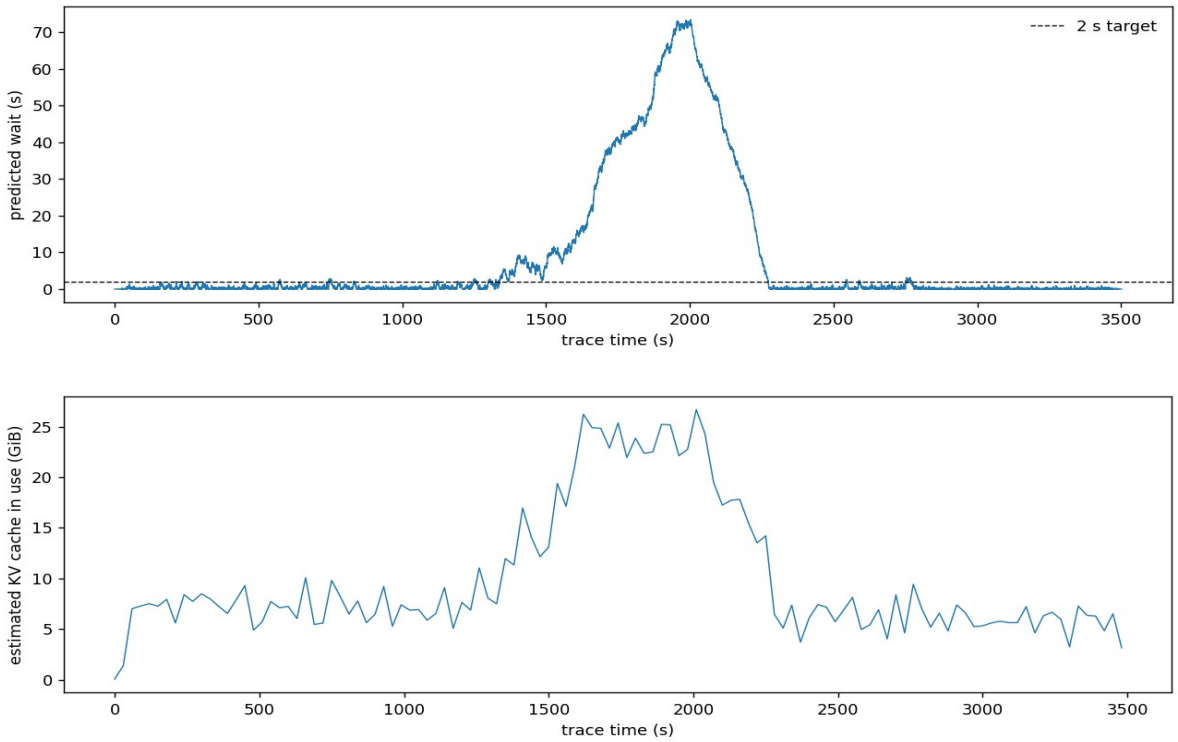
Full test suite (`bash scripts/run_tests.sh`):

```
collected 55 items
tests/test_memory_planner.py ......
tests/test_routing.py ..........
tests/test_sarathi_batch.py ................
tests/test_sarathi_chunk.py ......
tests/test_spf.py .......
tests/test_workload.py .......
55 passed in 0.73s
```

Part 1: Know the workload

A one-GPU forecast based on the trace itself (without Vidur) through constants in analysis/workload.py shows that there were 19,366 requests over 3,502 s and hence a basic QPS of 5.53 requests/s.





Q1. Does the model predict one GPU meets the 2 s target?

No. The projected P99 wait time is 72.1 seconds against the goal of 2 seconds, with 36.5% of requests actually waiting longer than 2 seconds. The target is missed dramatically in one big segment, t = 1192 seconds to t = 2272 seconds (1080 seconds long).

There are both more requests and longer prompts, mostly the former (the arrivals plot above). In the surge period (t = 1350 seconds to t = 2230 seconds), requests per second move from about 4.9 to around 7.3 (+47%), and the mean prompt increases by 16% (from 1,083 tokens to 1,256 tokens), so the number of prompt tokens per second peaks at 13,000, higher than the prefill capacity of 12,000 tokens/second, while the number of output tokens/second does not increase. The surge takes up 1.01 GPU-seconds of processing in a second (0.73 GPU-seconds outside of it), resulting in the queue building during the surge.

Q2. KV-cache memory: average, peak, and which GPU fits

Average 10.0 GiB, peak 26.7 GiB at t = 2010 s. Usable KV memory is 0.9 x GPU memory minus the 13.0 GiB of weights:

GPU	Usable for KV	Fits peak 26.7 GiB?	Fits average 10.0 GiB?
L4 (24 GiB)	8.6 GiB	no	no
A100 (40 GiB)	23.0 GiB	no	yes

GPU	Usable for KV	Fits peak 26.7 GiB?	Fits average 10.0 GiB?
L40S (48 GiB)	30.2 GiB	yes	yes
A100 / H100 (80 GiB)	59.0 GiB	yes	yes

When measuring by peak performance, the smallest GPU that can function is the L40S (48 GiB). However, when evaluating average performance, the A100 40 GiB is the most suitable GPU, although it would exhaust memory during the surge.

If the max_batch_size is (very) large, then the peak value is 94.4 GiB (at t = 1950 s), which no listed GPU can maintain. The capped peak (26.7 GiB) is a better measure, as a request that is in queue does not hold KV cache at that time, which means that the uncapped measure does not reflect the real memory consumption.

Q3. How sensitive is the wait to decode speed?

Baseline (30 ms step, batch 128): 4,267 tokens/s and P99 wait 72.1 s.

Decode setting	Throughput	Change	P99 wait	Change
step 20 ms, batch 128	6,400 tokens/s	+50%	30.7 s	-57%
step 50 ms, batch 128	2,560 tokens/s	-40%	201.4 s	+179%
step 30 ms, batch 64	2,133 tokens/s	-50%	386.4 s	+436%

P99 wait differs from throughput. When throughput goes down by 50% it becomes more than 5 times worse. The surge already takes 1.01 seconds of work per second, so even a minor change in throughput can lead to significant backlog.

As for decode_tok_per_s=float("inf"), it is still not enough with P99 wait clocking at 3.4 seconds. The limiting factor is in prefill: of 2,799 seconds of total work, 1,840 seconds must be in prompts that arrive at up to 13,000 tokens per second while delivery rate can be as high as 12,000 tokens per second.

Q4. Predictions recorded for Part 2

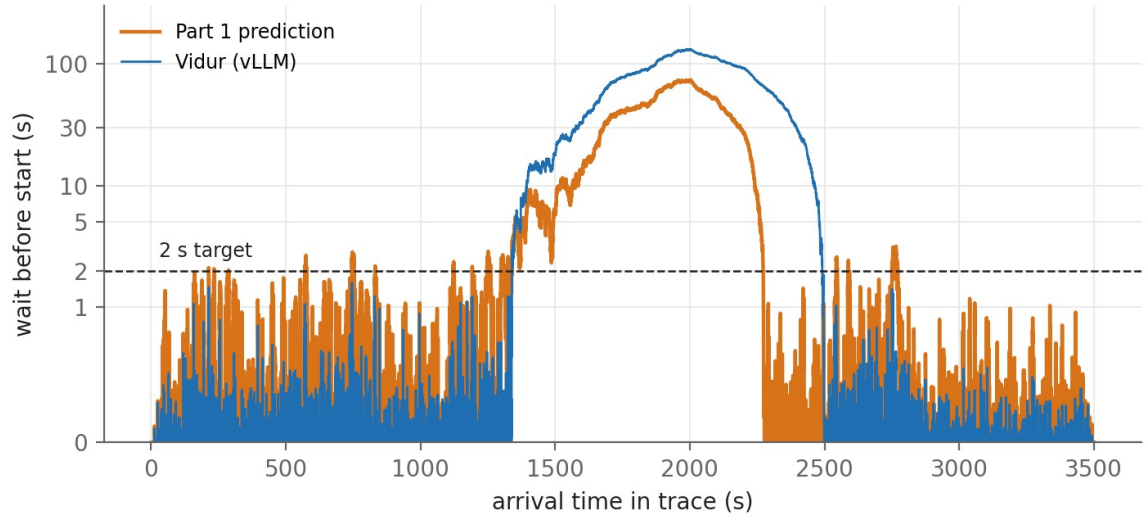
Prediction	Value
P99 wait	72.1 s
Longest stretch above the 2 s target	1192-2272 s (1080 s long)
Peak KV-cache memory	26.7 GiB (at t = 2010 s)

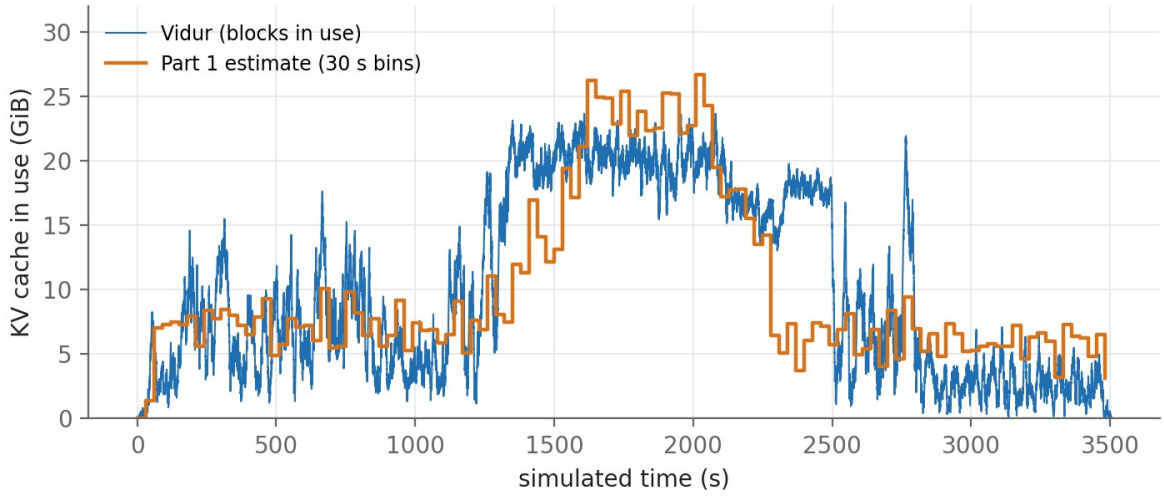
Part 2: Check the estimates

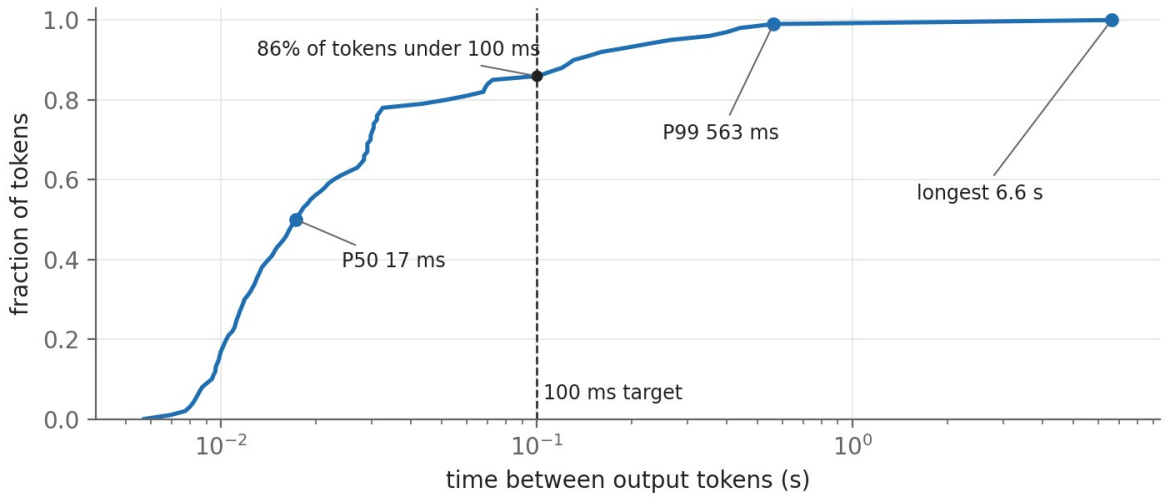
A single replay of the vLLM trace in regular time was conducted with the following parameters: QPS=5.53. However, the adventures below take place in a simulation.

Predicted vs measured

	Part 1 prediction	Vidur (vLLM)
P99 wait	72.1 s	**128.8 s**
Longest stretch above the 2 s target	1192–2272 s (1080 s long)	1340–2491 s (1152 s long)
Peak KV-cache memory	26.7 GiB (t = 2010 s)	23.8 GiB (t = 1952 s)







Q1. Which Part 1 predictions held?

Held: Both the prediction and Vidur managed to remain close to the target before the surge, and they went way above it for a long period (1080 seconds predicted - 1152 seconds measured), where peak usage of KV memory was indeed observed.

Missed: P99 wait was 72.1 seconds predicted versus 128.8 seconds (1.8x). The time above the target spikes ended 219 seconds later, while the predicted peak KV value is 12% higher than the measured one (26.7 GiB vs 23.8 GiB).

What the Part 1 model left out:

• Actual throughput is lower than optimal levels. The Part 1 model works with optimal figures (12,000 tokens per second and the whole decode batch of size 128) producing a workload of 2,799 seconds. Vidur spent about 3,495 seconds computing, as decoding batches are not full most of the time (the median number of requests in a batch equals 20).

• vLLM does not implement real-time decoding as it is busy working with prompts instead, meaning that it is up to 365 ms for a single batch iteration with no actual decoding taking place while the operation is performed, which cannot be said about the model.

• Memory allocation approaches are fundamentally different. The process in Part 1 reserves all memory upon request receiving; while Vidur uses only 16-token blocks, leading to lower peak memory usage and not allowing the scheduling process to be ever limited by memory issues.

Q2. Does vLLM meet the 100 ms target for time between tokens?

No. P50 is 17 ms, P99 563 ms (5.6 times and over the target of 100 ms) and the longest gap is 6.6 seconds; the normal token is fast and the problem is in the tail.

For the end user 6.6 seconds delay means that text stops midway showing as a hang, which is hidden by the median of 17 ms.

Q3. What causes the longest gaps?

In _get_next_batch, vLLM continuously adds every new request waiting for its entire prompt and checks the currently executing requests only if there is no match.

So any time a new waiting request fits, the iteration is done using all requests waiting for their prompts, while ignoring all decoding requests. Processing one batch with approximately 4000 prompt tokens takes around 365 ms, and the queue is never empty during peak times, so the iterations happen one after another: specifically, a 6.6 second time gap means about 18 iterations.

Q4. Predicting Part 3: time between tokens under a token budget C

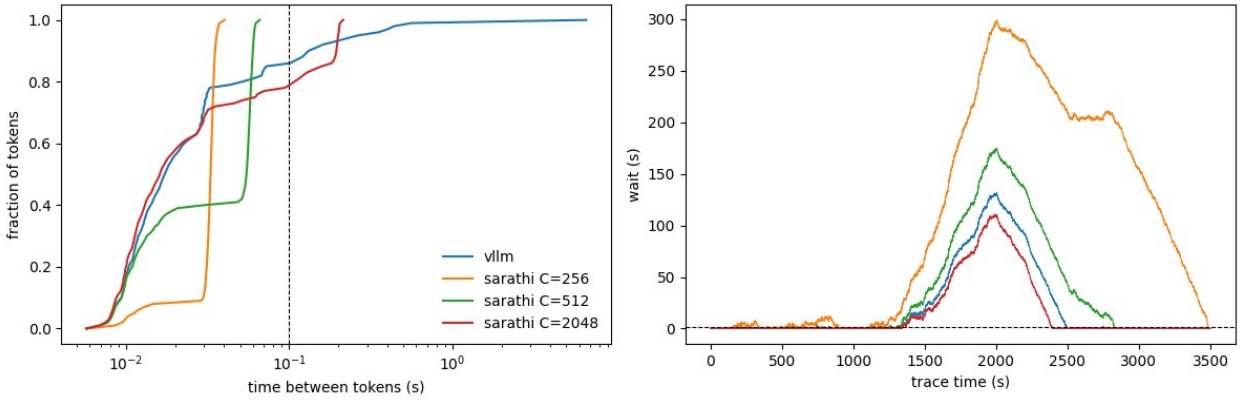
With budget C each running request receives its individual token every pass so the time between tokens is approximately T pass = 6 ms + C * 78 µs.

C	Predicted pass time (≈ P99 gap)	Meets 100 ms?	Model throughput C / (6 ms + C × 78 µs)
256	26.0 ms	yes	about 9,860 tokens/s
512	45.9 ms	yes	about 11,150 tokens/s
2048	165.7 ms	**no**	about 12,360 tokens/s

For C = 256 and C = 512 the requirements are satisfied; for C = 2048 they fail. The fixed part of 6 ms is incurred only once in every pass, therefore smaller packets will spend more time in this piece of time (for a typical prompt, 1,140 tokens will be processed in 5 passes with C = 256 and just in one pass with C = 2048). The throughput is decreasing as C gets smaller (approximately 9,860, 11,150 and 12,360 tokens/s), thus the queue is being exhausted much slower (lower C means smoother streaming but longer waiting time).

Part 3: Chunked prefill (Sarathi)

In three separate runs at C = 256, 512, 2048, Sarathi produced run timestamps 2026-10-27_09-32-40-989693, 2026-10-27_09-32-07-174520, and 2026-10-27_09-33-15-163902 as run identifiers. Running under parameters SCHED=sarathi, CHUNK=C, QPS=5.53, and invoking bash scripts/run_default.sh and using traces/azure_conversation.csv, compared with the Part 2 vLLM run, all three runs completed 19,366 requests.



Q1. Which budgets meet the 100 ms target?

C	Predicted P99 time between tokens (Part 2 Q4)	Measured P99	Meets 100 ms?
256	26.0 ms	37 ms	yes
512	45.9 ms	62 ms	yes
2048	165.7 ms	205 ms	no

The values C = 256 and C = 512 satisfy the goal of 100 ms; conversely, C = 2048 fails to meet this target, as I predicted in Part 2 Q4. Under the Sarathi model, the greatest latency is produced when using C = 256 (40 ms), C = 512 (66 ms), or C = 2048 (214 ms), whereas in the case of vLLM, the latency is much greater – over 6.6 s.

The reported values exceed my forecasts by up to 42%. The reported P99 latency is 37.2 ms, 61.5 ms, and 201.3 ms as compared to my predictions of 26.0 ms, 45.9 ms, 165.7 ms for each of the cases, respectively, which indicates that the model that allows us to calculate the cost of pass as 6 ms + 78 µs per token underestimates a full pass. The uncalculated expense is likely associated with the attention mechanism over the KV cache for all running requests.

Q2. Throughput, wait and the best budget

C	Model throughput (tokens/s)	Measured P99 wait	Median time to first token	Meets 100 ms?
256	9,858	293.2 s	100.82 s	yes
512	11,146	171.2 s	9.08 s	yes
2048	12,356	108.4 s	0.43 s	no

At C = 512, it achieves the objective with the minimum P99 wait of 171.2 seconds compared to 293.2 seconds at C = 256 (C = 2048 waits even less but fails to meet its goal with doing it in 205 ms). The median time required to deliver the first token

is 9.1 s while being 100.8 seconds at C = 256, as the queue at 256 gets jammed for almost the entire hour.

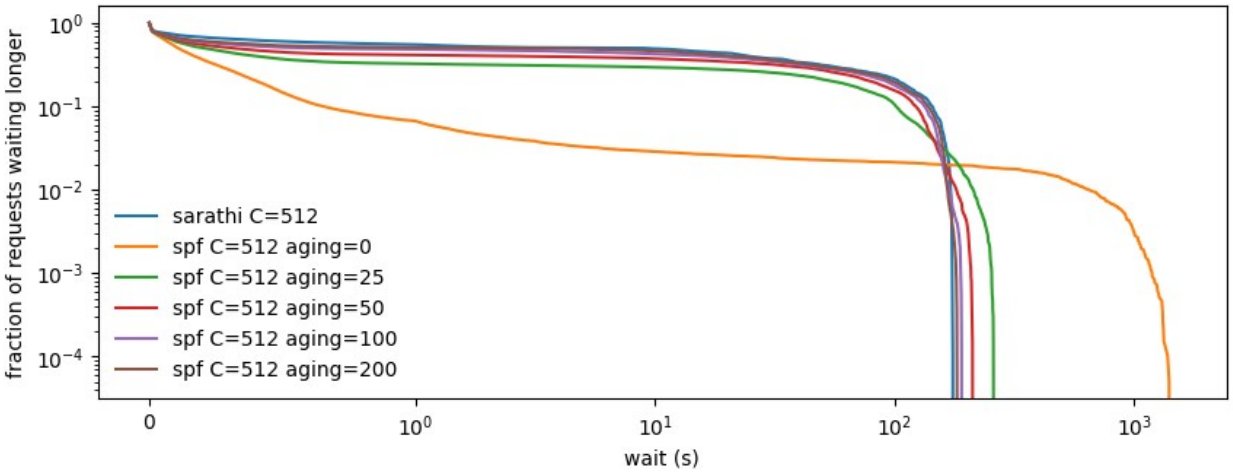
Having the smaller value of C negatively affects the throughput, since in this case the same messages have to go through more turns, while each turn incurs the fixed waiting time of 6 ms regardless of whether any tokens got carried.

Q3. Lowest wait, and can one GPU meet the 2 s target?

The minimum P99 waiting time is 108.4 s (C = 2048, which does surpass the 100 ms target); vLLM gives 128.8 s, C = 512 gives 171.2 s, and C = 256 gives 293.2 s. The prediction for Part 1 based on one GPU at full load is 72.1 s.

No, all runs are way above 2 s, and the best-case scenario in Part 1 (72.1 s) is already 36 x off: the surge needs 1.01 s of GPU work per second, so the queue forms regardless of what the scheduler does. Scheduling is just moving queue waiting, so additional hardware is needed to solve the problem.

Part 4: Which request goes first? (shortest prompt first, with aging)



Longest waits under aging 0 (shortest-first)

Arrived at (s)	Prompt tokens	Output tokens	Wait (s)
1344.0	4071	25	1482.5
1429.8	4071	25	1397.2
1346.4	4069	27	1377.0
1403.1	4069	27	1327.6
1495.7	4070	26	1324.0
1415.0	4069	27	1316.9

Arrived at (s)	Prompt tokens	Output tokens	Wait (s)
1428.2	4069	27	1313.7
1351.1	4068	28	1311.5

Wait by queue order

Order	Surge median wait	P99 wait	Worst wait	Requests waiting over 3 min
Arrival order (Part 3)	112.2 s	171.2 s	175.2 s	0
Aging 0 (plain shortest-first)	0.2 s	647.8 s	1482.5 s	383
Aging 25	38.3 s	212.5 s	258.7 s	425
Aging 50	78.4 s	181.1 s	211.7 s	202
Aging 100	96.7 s	168.9 s	190.5 s	84
Aging 200	105.5 s	166.2 s	182.1 s	27

Q1. Which order gives the lowest median and worst wait?

aging 0 had the lowest surge median wait at 0.2s. arrival order had lowest worst wait at 175.2s. aging 200 comes closest to this at 182.1 s. 2 s P99 target is met by nothing, the next best P99 is 182.1 s by aging 200.

Q2. Who waits longest under shortest-first, and why?

The longest wait were all at or nearly maximum-length with ~4070 tokens. They arrived at the start of the surge and run between 1311 to 1482 seconds. They wait long times because the surge brings short prompts that take higher priority so that the long prompts are starved for a very long time until the surge ends. Aging 0 has a much worse P99 than arrival order, since arrival order has 0 requests over P99, and Aging 0 has 383 requests over 3 minutes.

Q3. Does aging beat arrival order?

no aging beats the arrival order on both P99 and worst wait. Aging cuts the worst wait from 1482.5 s to ~182-259 s, and cuts P99 from 647.8 to 166-213s. It ends up affecting the surge median wait negatively. Aging reduces starvation once it becomes a valued enough to cause a long prompt to go in front of shorter prompts.

Part 5: More replicas, and where each request goes

Q1. How many replicas?

Replicas	Predicted P99 wait (Part 1 model)	Vidur P99 wait	Vidur P99 time between tokens
1	72.13 s	171.2 s (Part 3, C = 512)	62 ms
2	0.38 s	0.60 s	54 ms
3	0.15 s	—	—

2 replicas meets 2s P99 wait at 0.60s and 100 ms P99 TBT at 54 ms. One replica fails, and three replicas aren't needed. Both models agree that 1 fails and 2 passes. The model is 0.22s off.

Q2. Routers on the long-short bursts trace

Router	Wait P99	End-to-end P99	Busy % (replica 1 / 2)
Round-robin	8.99 s	9.78 s	100 / 5
LOR	1.14 s	1.98 s	80 / 76
LOT	0.77 s	0.85 s	81 / 81

LOT performed best at 81/81, round robin is worst with 100/5. For round robin, the trace alternates short and long prompts and round robin alternates between the 2 replicas, so one replica gets all of the long prompts and one gets all the short prompts. For LOR, it alternates like round robin when starting from empty, but then performs better later after there's already an imbalance. For FOT, it counts remaining tokens, so the load is more precis. It balances even from empty because it differentiates between small and long prompts.

Q3. Waiting-only LOR

the original LOR would pick replica 0 for almost every request. The waiting queues are almost always empty because requests move straight into the running set, so every replica shows a count of 0, they tie, and the tie goes to the lowest replica id. Counting waiting plus running requests is better, since the running requests are most of a replica's load, and a waiting-only count can't see a replica that is fully busy.

Q4. Round-robin vs LOT on the real trace

round robin gets a P99 wait of 0.60 s and LOT gets 0.40 s, and both have a P99 time between tokens of 54 ms. The router barely matters here because two replicas have plenty of spare capacity, and both are far under the targets. A load-aware router is worth it when request sizes are uneven or bursty and capacity is tight, like the bursts trace.